

University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 04

Experiment Title: Design and Simulation of a 4-Bit Arithmetic Logic Unit (ALU) Using VHDL

Submitted by:

Name: Nazmul Mridha
Student ID: 22201161
Section: C2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 29/10/2026

1. Experiment Name

Design and Simulation of a 4-Bit Arithmetic Logic Unit (ALU) with Carry/Borrow and Zero Flags Using VHDL in Xilinx ISE 14.7.

2. Objective

The main objectives of this experiment are:

1. To understand the working principle of an Arithmetic Logic Unit (ALU) as the computational core of a processor.
2. To design a 4-bit combinational ALU in VHDL with two 4-bit operands (X and Y), a 3-bit operation-select input (OPCODE), a 4-bit output F, a carry/borrow flag (COUT) and a zero flag (ZF).
3. To implement eight operations: ADD, SUB, AND, OR, XOR, NOT, INC and SHL.
4. To write a self-checking testbench that verifies the design with directed tests and an exhaustive test of all input combinations.
5. To generate the RTL schematic of the designed circuit in Xilinx ISE 14.7.
6. To simulate the design in ISim and examine the output for different input combinations.
7. To verify the functionality of the design using the ALU function table.

Design Description

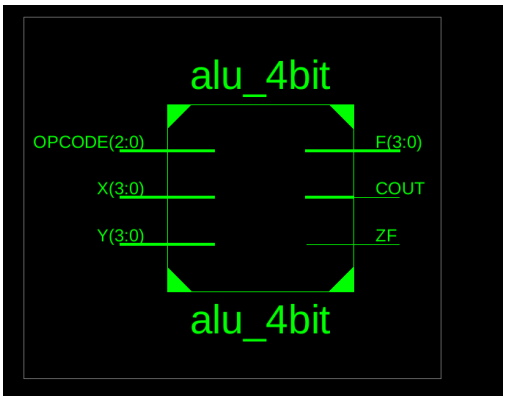
The ALU is a purely combinational circuit written in dataflow style. The inputs X and Y are converted to unsigned values and a 5-bit intermediate result r is computed according to OPCODE using a selected signal assignment. Bit 4 of r is the carry (ADD, INC), the borrow (SUB) or the bit shifted out (SHL), and it drives COUT. The lower four bits of r drive the output F. The zero flag ZF is set when F is 0000. The operations are summarized in the function table below.

OPCODE	Operation	F	COUT
000	ADD	X + Y	Carry out of bit 3
001	SUB	X - Y	Borrow (1 when X < Y)
010	AND	X and Y	0
011	OR	X or Y	0
100	XOR	X xor Y	0
101	NOT	not X	0
110	INC	X + 1	Carry out of bit 3
111	SHL	X shifted left by 1	Old X(3)

Table: Function table of the 4-bit ALU

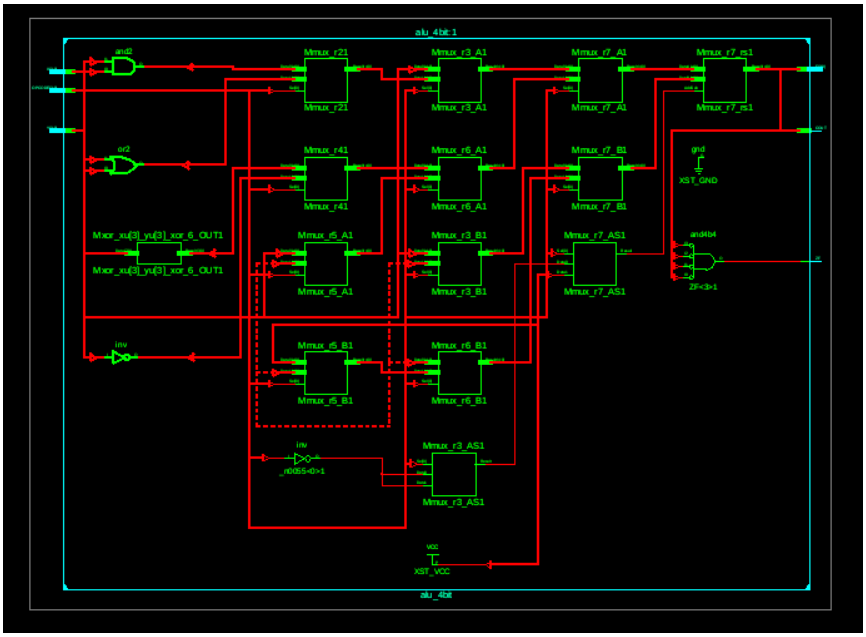
The zero flag is common to all operations: ZF = 1 when F = 0000, otherwise ZF = 0.

3. RTL-Schematic Diagrams



Top-Level RTL Schematic of the 4-Bit ALU (alu_4bit)

The top-level schematic shows the ports of the design: the 4-bit inputs X(3:0) and Y(3:0), the 3-bit operation-select input OPCODE(2:0), the 4-bit output F(3:0) and the single-bit outputs COUT and ZF.



Internal RTL Schematic of the 4-Bit ALU

The internal schematic shows how the synthesizer implemented the ALU. The AND, OR, XOR and NOT operations appear as the and2, or2, xor and inverter (inv) cells. The ADD, SUB and INC operations are built from the multiplexer/adder cells (Mmux_..._AS1), and the other multiplexer cells (Mmux_...) choose the final result according to OPCODE. An and4b4 gate with inverted inputs detects F = 0000 and drives the zero flag ZF, while the GND and VCC cells supply constant values. Being purely combinational, the design contains no flip-flops and no clock.

4. Code in github repository

https://github.com/Nazmul-SWE/VLSI

Design code (alu_4bit.vhd):

```
-- 4-bit ALU (dataflow style), Xilinx ISE / VHDL-93
-- OPCODE : 000 ADD   F = X + Y
--          001 SUB   F = X - Y   (COUT = borrow)
--          010 AND   F = X and Y
--          011 OR    F = X or Y
--          100 XOR   F = X xor Y
--          101 NOT   F = not X
--          110 INC   F = X + 1
--          111 SHL   F = X shifted left by 1 (COUT = old X(3))
-- ZF = '1' when F = "0000"
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity alu_4bit is
    Port ( X      : in  STD_LOGIC_VECTOR (3 downto 0);
           Y      : in  STD_LOGIC_VECTOR (3 downto 0);
           OPCODE : in  STD_LOGIC_VECTOR (2 downto 0);
           F      : out STD_LOGIC_VECTOR (3 downto 0);
           COUT   : out STD_LOGIC;
           ZF     : out STD_LOGIC);
end alu_4bit;


architecture Dataflow of alu_4bit is
    signal xu, yu : unsigned(3 downto 0);
    signal r      : unsigned(4 downto 0);   -- 5 bits: bit 4 = carry / borrow
begin
    xu <= unsigned(X);
    yu <= unsigned(Y);


    with OPCODE select
        r <= ('0' & xu) + ('0' & yu)   when "000",
             ('0' & xu) - ('0' & yu)   when "001",
             '0' & (xu and yu)         when "010",
             '0' & (xu or  yu)         when "011",
             '0' & (xu xor yu)         when "100",
             '0' & (not xu)            when "101",
             ('0' & xu) + 1            when "110",
             xu & '0'                  when "111",
             "00000"                   when others;


    F    <= std_logic_vector(r(3 downto 0));
    COUT <= r(4);
    ZF   <= '1' when r(3 downto 0) = "0000" else '0';
end Dataflow;
```

Testbench code (alu_4bit_tb.vhd):

```
-- Self-checking testbench for alu_4bit
-- Part 1: directed tests   Part 2: exhaustive test (8 opcodes x 16 x 16)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity alu_4bit_tb is
end alu_4bit_tb;


architecture Sim of alu_4bit_tb is


    component alu_4bit
        Port ( X      : in  STD_LOGIC_VECTOR (3 downto 0);
               Y      : in  STD_LOGIC_VECTOR (3 downto 0);
               OPCODE : in  STD_LOGIC_VECTOR (2 downto 0);
               F      : out STD_LOGIC_VECTOR (3 downto 0);
               COUT   : out STD_LOGIC;
               ZF     : out STD_LOGIC);
```

```
    end component;

    signal X_s, Y_s : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal OP_s     : STD_LOGIC_VECTOR (2 downto 0) := "000";
    signal F_s      : STD_LOGIC_VECTOR (3 downto 0);
    signal COUT_s, ZF_s : STD_LOGIC;

    -- expected 5-bit result (bit 4 = carry/borrow) from plain integer maths
    function model (x, y, op : integer) return integer is
        variable ux : unsigned(3 downto 0) := to_unsigned(x, 4);
        variable uy : unsigned(3 downto 0) := to_unsigned(y, 4);
    begin
        case op is
            when 0 => return x + y;
            when 1 => if x >= y then return x - y; else return x - y + 32; end if;
            when 2 => return to_integer(ux and uy);
            when 3 => return to_integer(ux or  uy);
            when 4 => return to_integer(ux xor uy);
            when 5 => return 15 - x;
            when 6 => return x + 1;
            when others => return 2 * x;
        end case;
    end function;

begin

    UUT : alu_4bit port map (X => X_s, Y => Y_s, OPCODE => OP_s,
                             F => F_s, COUT => COUT_s, ZF => ZF_s);

    process
        variable errors : integer := 0;
        variable total  : integer := 0;
        variable m      : integer;

        procedure drive (x, y, op : in integer) is
        begin
            X_s  <= std_logic_vector(to_unsigned(x, 4));
            Y_s  <= std_logic_vector(to_unsigned(y, 4));
            OP_s <= std_logic_vector(to_unsigned(op, 3));
            wait for 10 ns;
        end procedure;

        -- directed test with hand-calculated expected F, COUT, ZF
        procedure directed (x, y, op, ef : in integer; ec, ez : in STD_LOGIC) is
        begin
            drive(x, y, op);
            total := total + 1;
            if F_s /= std_logic_vector(to_unsigned(ef, 4)) or
               COUT_s /= ec or ZF_s /= ez then
                errors := errors + 1;
                report "Directed test failed: op=" & integer'image(op) &
                       " x=" & integer'image(x) & " y=" & integer'image(y)
                       severity error;
            end if;
        end procedure;
    begin
        wait for 10 ns;

        --        x   y  op  F   COUT ZF
        directed ( 5,  3, 0,  8, '0','0');   -- 5 + 3
        directed (15,  1, 0,  0, '1','1');   -- 15 + 1 : carry, zero
        directed ( 7,  2, 1,  5, '0','0');   -- 7 - 2
        directed ( 3,  5, 1, 14, '1','0');   -- 3 - 5  : borrow
        directed ( 6,  6, 1,  0, '0','1');   -- 6 - 6  : zero
        directed ( 9,  5, 2,  1, '0','0');   -- 1001 and 0101
        directed ( 9,  5, 3, 13, '0','0');   -- 1001 or  0101
        directed ( 9,  5, 4, 12, '0','0');   -- 1001 xor 0101
        directed (10,  0, 5,  5, '0','0');   -- not 1010
        directed (15,  0, 6,  0, '1','1');   -- 15 + 1
        directed ( 9,  0, 7,  2, '1','0');   -- 1001 << 1 = 0010, carry 1

        -- exhaustive check against the model
        for op in 0 to 7 loop
            for x in 0 to 15 loop
```

```
                for y in 0 to 15 loop
                    drive(x, y, op);
                    m := model(x, y, op);
                    total := total + 1;
                    if F_s /= std_logic_vector(to_unsigned(m mod 16, 4)) or
                       COUT_s /= std_logic(to_unsigned(m / 16, 1)(0)) or
                       (ZF_s = '1') /= ((m mod 16) = 0) then
                        errors := errors + 1;
                        report "Mismatch: op=" & integer'image(op) &
                               " x=" & integer'image(x) & " y=" & integer'image(y)
                               severity error;
                    end if;
                end loop;
            end loop;
        end loop;

        if errors = 0 then
            report "PASS: alu_4bit - " & integer'image(total) &
                   " tests, no errors" severity note;
        else
            report "FAIL: alu_4bit - " & integer'image(errors) &
                   " errors" severity error;
        end if;
        wait;
    end process;

end Sim;
```

5. Simulations



Figure: Design Simulation Waveform (ISim)

The testbench applies a new input combination every 10 ns and checks F, COUT and ZF at the end of each step. It first runs 11 directed tests whose expected values were calculated by hand, and then applies an exhaustive test of all 8 opcodes with all 16 × 16 combinations of X and Y (2048 checks), comparing the outputs with an integer-arithmetic reference model. In total 2059 checks are performed, and any mismatch is reported as an error message; at the end the testbench prints a PASS message when no error is found. In the waveform above, the directed tests occupy the first part of the simulation, after which the exhaustive test starts with OPCODE = 000 (ADD): X is held constant for 160 ns while Y sweeps from 0 to 15, and X then increments by one. The expected results of the directed tests, which match the simulated waveform, are listed below.

#	Time	Operation	X	Y	OPCODE	F	COUT	ZF
1	10 ns	ADD	0101	0011	000	1000	0	0
2	20 ns	ADD	1111	0001	000	0000	1	1
3	30 ns	SUB	0111	0010	001	0101	0	0
4	40 ns	SUB	0011	0101	001	1110	1	0
5	50 ns	SUB	0110	0110	001	0000	0	1
6	60 ns	AND	1001	0101	010	0001	0	0
7	70 ns	OR	1001	0101	011	1101	0	0
8	80 ns	XOR	1001	0101	100	1100	0	0
9	90 ns	NOT	1010	0000	101	0101	0	0
10	100 ns	INC	1111	0000	110	0000	1	1
11	110 ns	SHL	1001	0000	111	0010	1	0

Table: ALU Directed Test Sequence and Expected Output

The simulated waveform follows the function table: ADD produces the sum with the carry flag, SUB produces the difference with the borrow flag, the logical operations produce the expected bitwise results, INC and SHL produce the incremented and shifted values with COUT set when a bit is carried out, and ZF is set whenever F is 0000.

6. Conclusion

In this experiment, a 4-bit ALU supporting eight arithmetic and logical operations, together with a carry/borrow flag and a zero flag, was designed in VHDL using Xilinx ISE 14.7. The RTL schematic shows that the design is implemented as purely combinational logic made of gates, adder/multiplexer cells and a zero-detect gate. Behavioral simulation in ISim with a self-checking testbench confirmed that every operation produces the correct output and that the COUT and ZF flags behave as specified. Since the simulated outputs match the expected function table, the design is considered functionally verified. This experiment provided practical experience in VHDL programming, combinational circuit design, self-checking testbenches and digital logic simulation, and the ALU is a fundamental building block of processors and computing systems.